

Group : G4

Contributors

- | | |
|--------------------------|--------------|
| 1. Mr. Thananop Kullapan | 653-01821-21 |
| 2. Mr. Kanawat Suwandee | 687-00268-21 |
| 3. Mrs. Yuwadee Tongkong | 687-20736-21 |
-

Goal:

- can implement and configure the clustering algorithms.
- can provide the reasons why the chosen algorithm is the most suitable for given datasets.
- can provide the answer with evidence.

Your tasks are:

1. Explore the given datasets **dataset A** and **dataset B**. At first, you should try different clustering algorithms (k-mean, GMM, and DBSCAN) on **dataset A**. We have provided parts of the implementation in [Github Lab5](#). The hyperparameters were only roughly set up, so you may need to find the most suitable setting on your own. Also, we provided the ground truth clusters in the attribute called 'label' in the dictionary (for both dataset A and B).

Then, answer the following questions:

- a. When the number of clusters is unknown, what is the most suitable algorithm (including the hyperparameter settings) ? *Also, provide the reasons and evidence to support why the chosen algorithm is the most suitable for **dataset A**.*
 - b. However, when the number of clusters is known, what is the most suitable algorithm (including the hyperparameter settings) ? *Provide the reasons and evidence to support why the chosen algorithm is the most suitable for **dataset A**.*
2. If you switch to **dataset B**, would you get the same answers? And why?
-

Question 1: Explore dataset A

1.1 When the number of clusters is **unknown**, what is the most suitable algorithm (including the hyperparameter settings) ? Also, provide the reasons and evidence to support why the chosen algorithm is the most suitable for **dataset A**.

Answer

เหตุผลที่เลือกใช้ DBSCAN สำหรับ Dataset A

สาเหตุหลักที่ DBSCAN ตอบโจทย์ Dataset A มากที่สุด เป็นเพราะอัลกอริทึมนี้ใช้วิธีประเมินความหนาแน่นของข้อมูลในการจัดกลุ่ม ทำให้ไม่ต้องระบุจำนวนกลุ่มล่วงหน้า (ต่างจาก K-Means ที่ต้องกำหนดค่า k) นอกจากนี้ ข้อมูลภาพที่ผ่านการลดมิติด้วย PCA มักมีการกระจายตัวที่มีรูปร่างซับซ้อนและไม่แน่นอน DBSCAN สามารถกวาดจับกลุ่มข้อมูลที่มีรูปร่างอิสระเหล่านี้ได้ดีเยี่ยม และยังมีกลไกในการคัดกรองจุดข้อมูลที่อยู่โดดเดี่ยวหรือมีความหนาแน่นต่ำออกไปเป็นจุดรบกวน (Noise ซึ่งในโค้ดจะแทนด้วยค่า -1) ทำให้การจัดกลุ่มมีความแม่นยำและไม่ถูกบิดเบือนจาก Outliers

การค้นหามิติที่เหมาะสมผ่านโค้ดส่วนแรก

ในโค้ดส่วนแรกจะเป็นการค้นหามิติที่ดีที่สุด โดย Grid Search เนื่องจาก DBSCAN ต้องการพารามิเตอร์สำคัญ 2 ตัวคือ **eps** (รัศมีความหนาแน่น) และ **min_samples** (จำนวนจุดขั้นต่ำในการสร้างกลุ่ม) จึงเริ่มต้นด้วยการกำหนดช่วงของตัวแปรทั้งสอง จากนั้นใช้ลูปซ้อนลูปเพื่อนำแต่ละคู่พารามิเตอร์ไปเทรนโมเดล DBSCAN เพื่อดูผลลัพธ์การคาดการณ์ (**predicted_clusters**) โค้ดได้ตั้งเงื่อนไขไว้ว่า หากโมเดลแบ่งกลุ่มได้น้อยกว่า 2 กลุ่ม (เช่น มองทุกอย่างเป็นกลุ่มเดียวหรือเป็น Noise หมด) จะถูกปิดตกให้คะแนนเป็น -1 แต่หากแบ่งได้มากกว่านั้น จะนำผลไปวัดประสิทธิภาพด้วย **Adjusted Rand Score (ARI)**

The ARI Formula

The ARI is calculated by adjusting the raw Rand Index (RI) based on the expected similarity of random clusterings:  OECD AI Policy Observatory

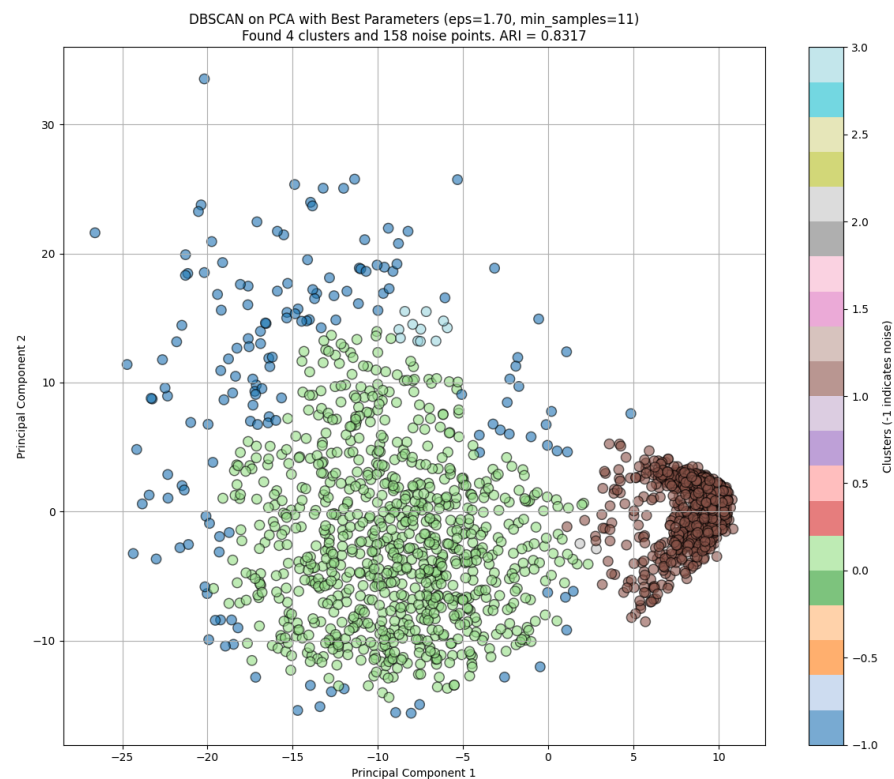
$$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]}$$

Where:

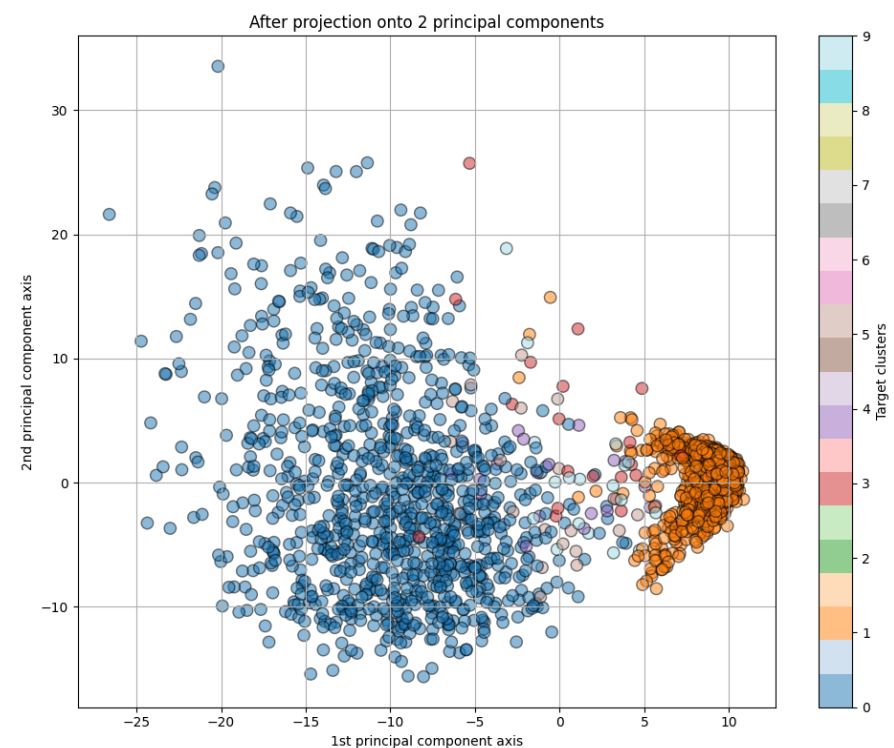
- **RI (Rand Index):** $\frac{a+b}{a+b+c+d}$ (Percentage of correct pair assignments).
- **E[RI]:** The expected RI for random labels.
- **max(RI):** The maximum possible RI value (1.0). 

เหตุผลที่เราใช้ ARI เป็นเกณฑ์ตัดสินใจ เนื่องจาก Dataset A มีคลาสจริง (**true_labels**) ให้เปรียบเทียบ ค่า ARI จะช่วยประเมินความแม่นยำว่ากลุ่มที่โมเดลสร้างขึ้นตรงกับความเป็นจริงมากน้อยแค่ไหน (ค่าเข้าใกล้ 1 คือดีที่สุด) โดยจะประเมินและเก็บค่าพารามิเตอร์ที่ทำคะแนน ARI ได้สูงสุดไว้เพื่อนำไปใช้งานต่อไป

การประเมินผลลัพธ์



ภาพที่ 1 ผลลัพธ์จากการรันโค้ดโดยใช้ Algorithm DBSCAN บน Dataset A



ภาพที่ 2 ผลลัพธ์จากการรันโค้ดโดยเป็นข้อมูลจริง บน Dataset A

เมื่อผ่านการค้นหาจนได้ `best_eps` และ `best_min_samples` มาแล้ว ในโค้ดส่วนที่สอง จะนำค่าที่ดีที่สุดนี้มาเทรนโมเดล DBSCAN อีกครั้งเพื่อเตรียมแสดงผล โค้ดส่วนนี้ทำหน้าที่สร้างกราฟ Scatter Plot บนแกน Principal Component ทั้งสองแกน กราฟจะทำการลงสีแบ่งแยกกลุ่มที่โมเดลค้นพบอย่างชัดเจน ทำให้เราสามารถตรวจสอบได้ทันทีว่าโมเดลแบ่งข้อมูลออกเป็นกี่กลุ่ม (`n_clusters_best`) และมีจุดไหนบ้างที่ถูกคัดออกเป็นจุดรบกวน (`n_noise_best`) ข้อมูลทั้งหมดนี้ประกอบกับตารางแสดงผลคะแนน ARI ที่สูงที่สุด โดยตารางที่แนบมาด้านล่างนี้ ได้แสดง 5 อันดับแรกที่มีค่า ARI สูงที่สุด

Iteration	eps	min_samples	ค่า ARI
165	1.7	11	0.8316994
151	1.6	10	0.8270263
166	1.7	12	0.8269256
50	0.8	13	0.8266362
137	1.5	9	0.8209748

หมายเหตุเพิ่มเติม

หากสังเกตจากกราฟพล็อตที่ได้ โมเดล DBSCAN สามารถค้นพบกลุ่มข้อมูลเพียง 4 กลุ่ม (สีน้ำเงิน, เขียว, ฟ้าอ่อน และน้ำตาล) ซึ่งไม่ตรงกับจำนวนคลาสที่แท้จริงที่มีถึง 6 คลาส อย่างไรก็ตามพล็อตนี้ไม่ใช่ความผิดพลาดของโมเดล แต่เป็นภาพสะท้อนที่สมเหตุสมผลตามกลไกของอัลกอริทึม เมื่อพิจารณาจากโครงสร้างของชุดข้อมูล

เนื่องจาก Dataset A มีความไม่สมดุลของคลาส (Imbalanced Data) ค่อนข้างสูง โดยคลาสหลัก 2 คลาสมีข้อมูลรวมกันมากกว่า 2,000 ตัว ในขณะที่คลาสน้อยอีก 4 คลาสมีข้อมูลเพียงคลาสละ 20 ตัว เมื่อข้อมูลถูกลดมิติด้วย PCA ลงมาเหลือ 2 มิติ คลาสน้อยเหล่านี้จึงมีการกระจายตัวที่เบาบางมากจนความหนาแน่นไม่ผ่านเกณฑ์ขั้นต่ำ (`min_samples = 11`) ที่โมเดลเรียนรู้มาว่าดีที่สุด จุดข้อมูลขนาดเล็กเหล่านี้จึงถูกอัลกอริทึมจัดให้เป็นจุดรบกวน (Noise: -1) หรือถูกรวบเข้ากับกลุ่มใหญ่ที่ซ้อนทับกันอยู่

```
labels, counts = np.unique(image_with_label['label'], return_counts=True)

print(labels) # [0 1 3 4 5 9]
print(counts) # [980 1135 20 20 20 20]
```

ดังนั้น การที่โมเดลค้นพบ 4 กลุ่มหลัก จึงเป็นการทำงานที่ถูกต้องตามหลักการวิเคราะห์ความหนาแน่น (Density-Based) และการที่ค่า ARI ออกมาสูงถึง 0.8317 ก็เป็นตัวยืนยันว่า โมเดลสามารถจับแพทเทิร์นโครงสร้างหลักส่วนใหญ่ของข้อมูลชุดนี้ได้อย่างแม่นยำแล้วภายใต้ข้อจำกัดของมิติข้อมูลที่มีอยู่

HW5: Clustering Algorithm

Python Code (ส่วนท้ายสุดของจากที่มีอยู่บน main.ipynb)

```
from sklearn.metrics import adjusted_rand_score
from sklearn.cluster import DBSCAN
import numpy as np
import pandas as pd

X = data_array_pca
true_labels = image_with_label['label']

# Define the ranges for eps and min_samples to search
eps_range = np.arange(0.5, 5.0, 0.1)
min_samples_range = range(2, 15, 1)

best_ari = -1
best_eps = None
best_min_samples = None
all_results = []

print("Starting Grid Search for DBSCAN hyperparameters...")

for eps in eps_range:
    for min_samples in min_samples_range:
        dbscan = DBSCAN(eps=eps, min_samples=min_samples).fit(X)
        predicted_clusters = dbscan.labels_

        # Ignore cases where all points are noise (-1) or only one cluster is found (excluding noise)
        n_clusters = len(set(predicted_clusters)) - (1 if -1 in predicted_clusters else 0)
        if n_clusters < 2: # We expect at least two clusters to make sense for ARI comparison
            ari = -1 # Assign a low score if clustering is trivial
        else:
            ari = adjusted_rand_score(true_labels, predicted_clusters)

        all_results.append({'eps': eps, 'min_samples': min_samples, 'ari': ari})

        if ari > best_ari:
            best_ari = ari
            best_eps = eps
            best_min_samples = min_samples

results_df = pd.DataFrame(all_results)

print("Grid Search Complete.")
print(f"Best ARI: {best_ari:.4f}")
print(f"Best eps: {best_eps:.2f}")
print(f"Best min_samples: {best_min_samples}")

#-----
# Visualization
#-----

plt.figure(figsize=(12, 10))

# Run DBSCAN with the best parameters found
dbscan_best = DBSCAN(eps=best_eps, min_samples=best_min_samples).fit(X)
predicted_cluster_best = dbscan_best.labels_

n_clusters_best = len(set(predicted_cluster_best)) - (1 if -1 in predicted_cluster_best else 0)
n_noise_best = list(predicted_cluster_best).count(-1)

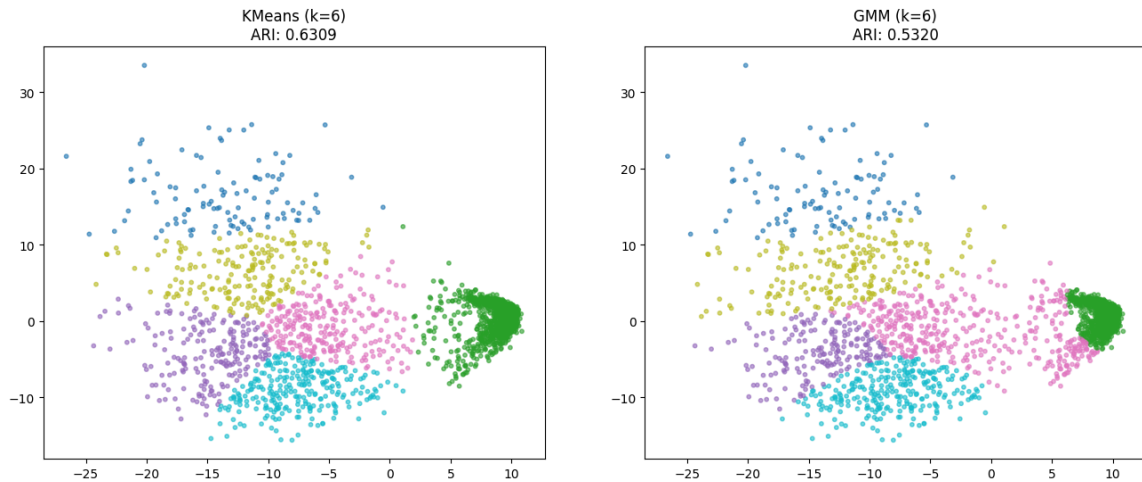
plt.scatter(data_array_pca[:, 0], data_array_pca[:, 1], c=predicted_cluster_best, cmap="tab20", edgecolor='k', s=80, alpha=0.6)
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title(f'DBSCAN on PCA with Best Parameters (eps={best_eps:.2f}, min_samples={best_min_samples})\nFound {n_clusters_best} clusters and {n_noise_best} noise points. ARI = {best_ari:.4f}')

plt.colorbar(label='Clusters (-1 indicates noise)')
plt.grid()
plt.tight_layout()
plt.show()

# Display the top 5 results from the grid search for more context
print("\nTop 5 results by ARI:")
display(results_df.sort_values(by='ari', ascending=False).head(5))
```

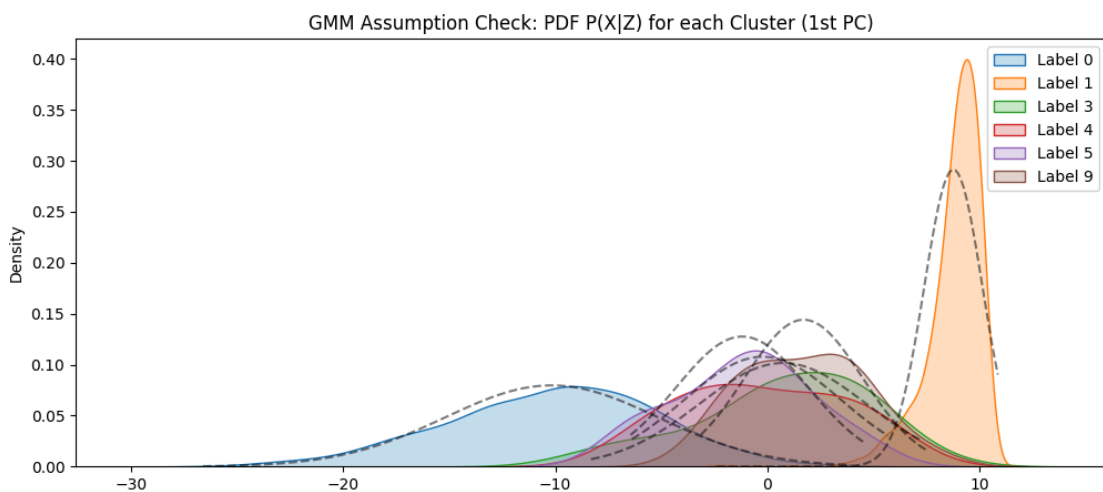
1.2 However, when the number of clusters is **known**, what is the most suitable algorithm (including the hyperparameter settings) ? *Provide the reasons and evidence to support why the chosen algorithm is the most suitable for dataset A.*

Answer



ภาพที่ 3 ผลลัพธ์จากการรันโค้ดการ Clustering ระหว่าง KMeans และ GMM บน Dataset A

เมื่อทราบจำนวนกลุ่มที่แน่ชัดคือ $k = 6$ อัลกอริทึมที่เหมาะสมที่สุดสำหรับ Dataset A คือ **K-Means** โดยมีการตั้งค่าพารามิเตอร์ที่เหมาะสมคือ **n_clusters=6** ตามจำนวนคลาสที่ทราบล่วงหน้า ร่วมกับการใช้ **n_init='auto'** เพื่อให้อัลกอริทึมเลือกจุดเริ่มต้นโดยไม่มีการอคติ และกำหนด **random_state=0** เพื่อให้ผลลัพธ์สามารถทำซ้ำและตรวจสอบได้ การตัดสินใจเลือก K-Means แทนที่จะเป็นอัลกอริทึมอย่าง Gaussian Mixture Model (GMM) มีที่มาจากหลักฐานทั้งด้านข้อสมมติฐานทางสถิติและผลการประเมินประสิทธิภาพ ARI



ภาพที่ 4 ผลลัพธ์จากการรันโค้ดเพื่อตรวจสอบสมมติฐานของ GMM บน Dataset A

ก่อนการประเมินประสิทธิภาพ ได้มีการตรวจสอบข้อสมมติฐานพื้นฐานของอัลกอริทึม (Assumption Check) โดยเฉพาะโมเดล GMM ที่มีข้อสมมติฐานหลักว่าข้อมูลในแต่ละกลุ่มย่อยจะ

ต้องมีการกระจายตัวแบบปกติ ซึ่งเมื่อพิจารณาจากกราฟฟังก์ชันความหนาแน่น (Density Function - PDF) ของข้อมูลแต่ละคลาสบนแกน Principal Component ที่ 1 พบว่าการกระจายตัวจริงของข้อมูล (เส้นทึบ) โดยเฉพาะในคลาส 0 และ 1 มีลักษณะเบ้และไม่ได้แนบสนิทไปกับรูปทรงระฆังคว่ำแบบอุดมคติ (เส้นประสีดำ) การที่โครงสร้างข้อมูลจริงละเมิดข้อสมมติฐานความน่าจะเป็นนี้ เป็นสัญญาณบ่งชี้ที่ชัดเจนว่า GMM จะทำงานกับข้อมูลชุดนี้ได้ไม่เต็มประสิทธิภาพ หรือไม่น่าเชื่อถือเท่าที่ควร

เมื่อนำอัลกอริทึมทั้งสองมาทดสอบการจัดกลุ่มข้อมูลจริง ผลลัพธ์เชิงปริมาณได้ออกมายืนยันสมมติฐานดังกล่าว โดยโมเดล K-Means สามารถทำคะแนน ARI ได้ถึง 0.6309 ซึ่งสูงกว่า GMM ที่ทำได้เพียง 0.5320 สาเหตุที่เป็นเช่นนี้เนื่องจาก K-Means อาศัยหลักการวัดระยะห่างจากจุดกึ่งกลาง (Centroid-based) เพื่อแบ่งพื้นที่ของข้อมูล ซึ่งสอดคล้องกับธรรมชาติการกระจายตัวของข้อมูล PCA ในชุดนี้ได้ดีกว่าการพยายามจับกลุ่มด้วยรูปทรงความน่าจะเป็นแบบวงรีของ GMM ที่ได้รับผลกระทบจากการกระจายตัวที่ไม่เป็นแบบปกติ หลักฐานทั้งหมดนี้จึงเป็นการสนับสนุนอย่างหนักแน่นว่า K-Means เป็นตัวเลือกที่เหมาะสมที่สุดสำหรับสถานการณ์นี้

หมายเหตุเพิ่มเติม

ด้วยลักษณะเช่นเดียวกับ DBSCAN ทั้งจาก Imbalanced Data และการลดขนาดข้อมูลโดย PCA ทำให้ Algorithm ทำให้สูญเสียความแปรปรวนหรือคุณลักษณะบางอย่างที่ใช้แยกแยะคลาstry่อย จุดข้อมูลของคลาstryขนาดเล็กจึงถูกฉายลงมาซ้อนทับ (Overlap) หรือกลืนไปกับกลุ่มข้อมูลหลัก ทำให้ก้อนข้อมูลหลักที่ควรจะเป็นกลุ่มเดียวถูกแยกย่อยออกเป็นหลายสืออย่างผิดเพี้ยน

HW5: Clustering Algorithm

Python Code (ส่วนท้ายสุดของจากที่มีอยู่บน main.ipynb)

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.metrics import adjusted_rand_score
from scipy.stats import norm

# 1. Setup Data & Known k
X = data_array_pca
true_labels = image_with_label['label']
k_known = len(np.unique(true_labels))

# --- 2. Check GMM Assumption (Normality of clusters) ---
plt.figure(figsize=(12, 5))
for label in np.unique(true_labels):
    subset = X[true_labels == label, 0]
    sns.kdeplot(subset, label=f'Label {label}', fill=True)

    # Overlay ideal Normal distribution for comparison
    mu, std = norm.fit(subset)
    x_range = np.linspace(subset.min(), subset.max(), 100)
    plt.plot(x_range, norm.pdf(x_range, mu, std), color='black', linestyle='--', alpha=0.5)

plt.title("GMM Assumption Check: PDF P(X|Z) for each Cluster (1st PC)")
plt.legend()
plt.show()

# --- 3. Run Algorithms ---
# KMeans
kmeans = KMeans(n_clusters=k_known, random_state=0, n_init='auto').fit(X)
km_labels = kmeans.labels_
km_ari = adjusted_rand_score(true_labels, km_labels)

# GMM
gm = GaussianMixture(n_components=k_known, random_state=0).fit(X)
gm_labels = gm.predict(X)
gm_ari = adjusted_rand_score(true_labels, gm_labels)

# --- 4. Compare results ---
print(f"[Result] KMeans ARI: {km_ari:.4f}")
print(f"[Result] GMM ARI: {gm_ari:.4f}")

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

ax1.scatter(X[:, 0], X[:, 1], c=km_labels, cmap='tab10', s=10, alpha=0.6)
ax1.set_title(f"KMeans (k={k_known})\nARI: {km_ari:.4f}")

ax2.scatter(X[:, 0], X[:, 1], c=gm_labels, cmap='tab10', s=10, alpha=0.6)
ax2.set_title(f"GMM (k={k_known})\nARI: {gm_ari:.4f}")

plt.show()

# Reasoning for suitability
print("Reasoning: ")
if gm_ari > km_ari:
    print("GMM is more suitable as it accounts for elliptical cluster shapes (covariance) which we see in the PCA plot.")
else:
    print("KMeans is suitable due to its simplicity and computational efficiency if clusters are roughly spherical.")
```


Question 2: Switch to dataset B

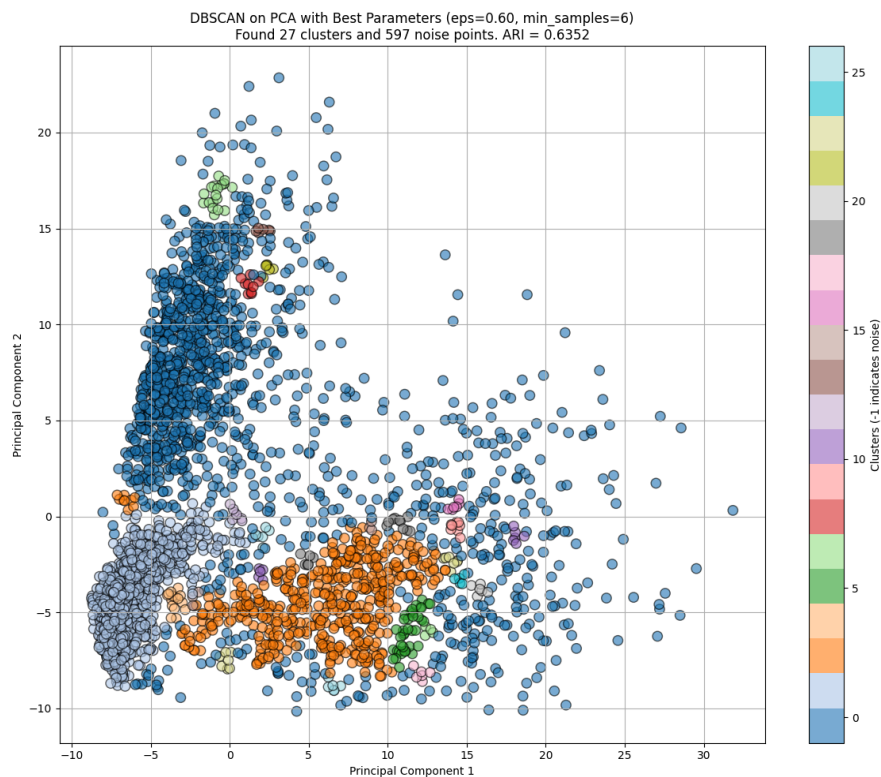
2.1 When the number of clusters is **unknown**, what is the most suitable algorithm (including the hyperparameter settings) ? Also, provide the reasons and evidence to support why the chosen algorithm is the most suitable for **dataset A**.

Answer

สำหรับคำถามข้อ 2.1 เมื่อไม่ทราบจำนวนกลุ่มที่แน่ชัด อัลกอริทึมที่เหมาะสมที่สุดสำหรับ Dataset B ยังคงเป็น **DBSCAN** ครับ โดยใช้หลักการและเหตุผลสนับสนุนแบบเดียวกับการวิเคราะห์ Dataset A ทุกประการ ไม่ว่าจะเป็นเรื่องของการไม่ต้องระบุค่า k ล่วงหน้า การรองรับโครงสร้างข้อมูลแบบอิสระ และกลไกคัดกรองจุดรบกวน (Noise)

พารามิเตอร์ที่เหมาะสมที่สุดและหลักฐานสนับสนุน

ผลลัพธ์จากการทำ Grid Search และการใช้เกณฑ์ ARI เป็นตัวชี้วัด ยืนยันว่าการตั้งค่าพารามิเตอร์ที่ดีที่สุดคือ **eps=0.6** และ **min_samples=6** ซึ่งสามารถทำคะแนน ARI ได้สูงสุดที่ **0.6352**

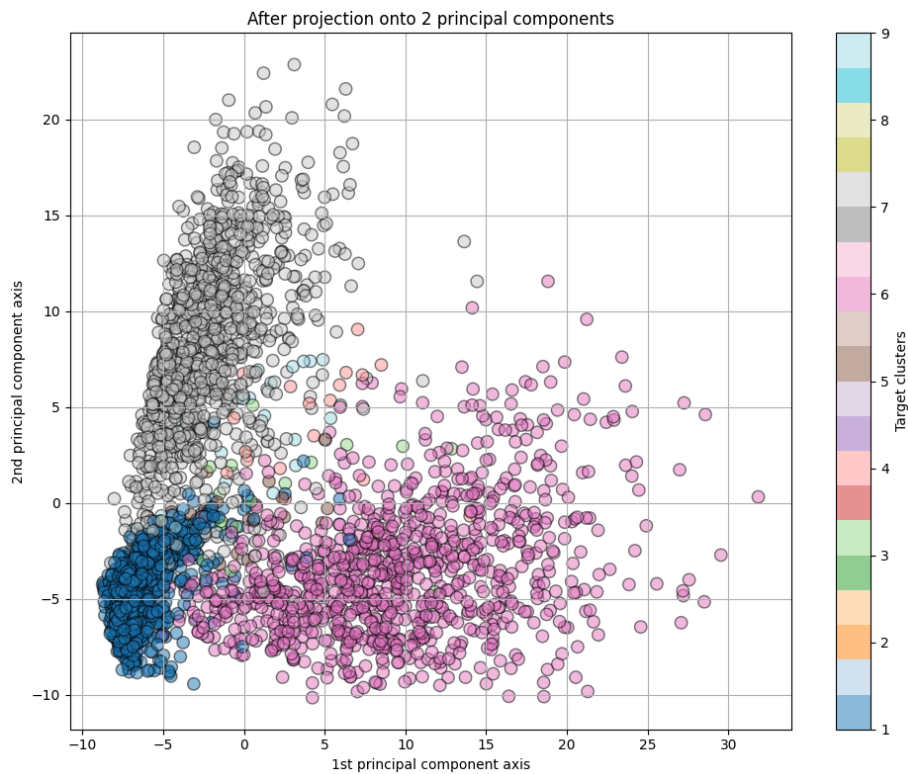


ภาพที่ 5 ผลลัพธ์จากการรันโค้ดโดยใช้ Algorithm DBSCAN บน Dataset B

Iteration	eps	min_samples	ค่า ARI
17	0.6	6	0.635227
33	0.7	9	0.634423

HW5: Clustering Algorithm

49	0.8	12	0.632300
18	0.6	7	0.627842
2	0.5	4	0.617400



ภาพที่ 6 ผลลัพธ์จากการรันโค้ดโดยเป็นข้อมูลจริง บน Dataset B

หมายเหตุเพิ่มเติม

จุดที่น่าสนใจของ Dataset B คือโมเดลค้นพบกลุ่มย่อยถึง 27 กลุ่ม พร้อมกับมีจุดรบกวน (Noise) 597 จุด ซึ่งมองเผินๆ อาจจะดูห่างไกลจากความจริงที่มีเพียง 7 คลาส แต่ในทางสถิติแล้ว นี่คือภาพสะท้อนที่ถูกต้องที่สุดของโครงสร้างข้อมูลชุดนี้

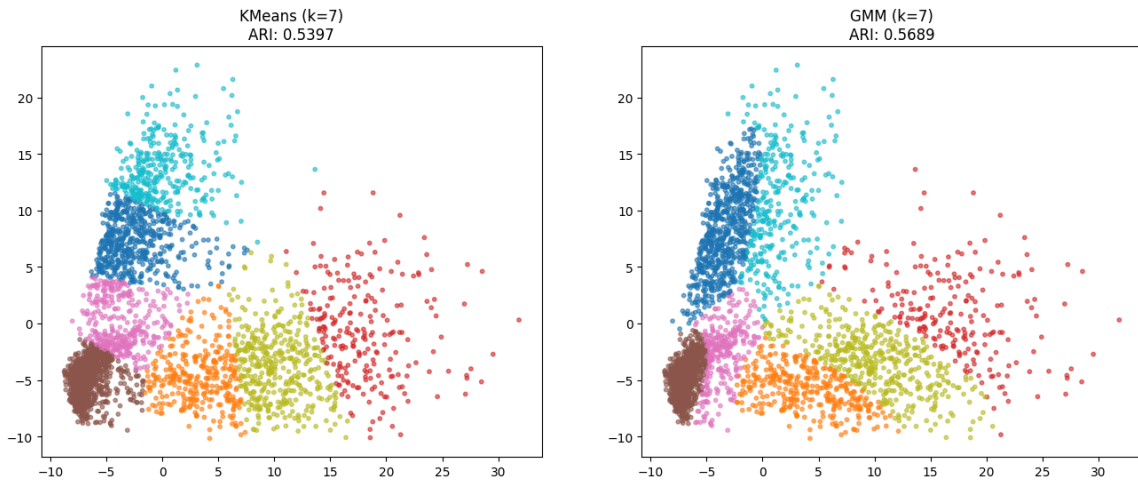
Dataset B มีลักษณะข้อมูลที่ไม่สมดุล (Imbalanced Data) อย่างรุนแรง แต่น้อยกว่า Dataset A โดยมีคลาสหลักขนาดใหญ่ 3 คลาส (จำนวน 1,135, 958 และ 1,028 ตัว) และคลาสย่อยขนาดเล็กอีก 4 คลาส (คลาสละ 20 ตัว)

```
labels, counts = np.unique(image_with_label['label'], return_counts=True)

print(labels) # [1 3 4 5 6 7 9]
print(counts) # [1135 20 20 20 958 1028 20]
```

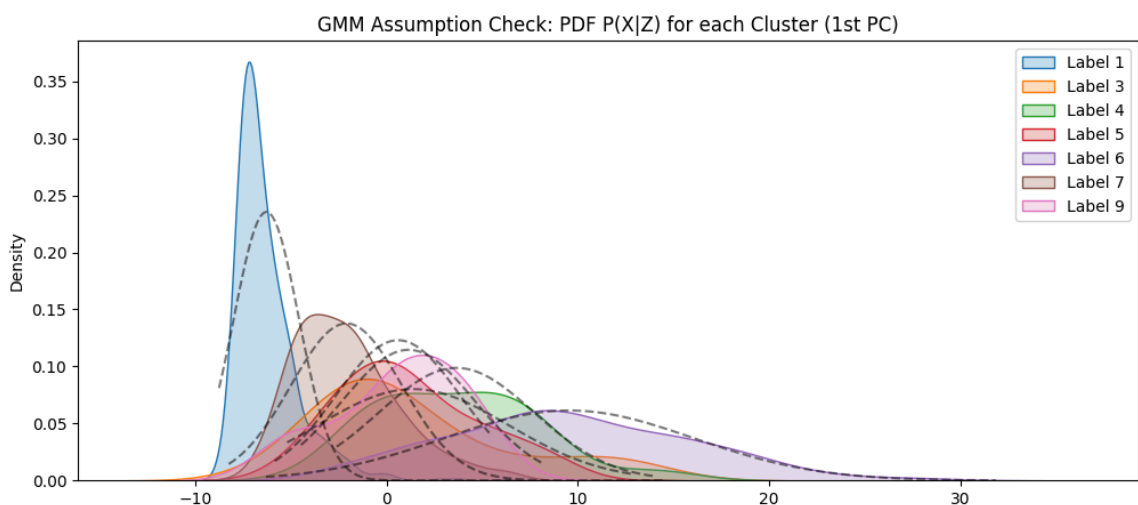
2.2 However, when the number of clusters is **known**, what is the most suitable algorithm (including the hyperparameter settings) ? *Provide the reasons and evidence to support why the chosen algorithm is the most suitable for dataset B.*

Answer



ภาพที่ 7 ผลลัพธ์จากการรันโค้ดการ Clustering ระหว่าง KMeans และ GMM บน Dataset B

เมื่อทราบจำนวนกลุ่มที่แน่ชัดคือ $k = 7$ สำหรับ Dataset B อัลกอริทึมที่เหมาะสมที่สุดจะเปลี่ยนมาเป็น Gaussian Mixture Model (GMM) โดยมีการตั้งค่าพารามิเตอร์ที่สำคัญคือ `n_components=7` (ตามจำนวนคลาสที่ทราบล่วงหน้า) และกำหนด `random_state=0` เพื่อให้ผลลัพธ์สามารถทำซ้ำได้ การตัดสินใจเลือก GMM ในกรณีนี้มีเหตุผลสนับสนุนที่ชัดเจนจากทั้งลักษณะการกระจายตัวทางสถิติและคะแนนประเมินประสิทธิภาพที่เหนือกว่า



ภาพที่ 8 ผลลัพธ์จากการรันโค้ดเพื่อตรวจสอบสมมติฐานของ GMM บน Dataset B

จากหลักฐานการตรวจสอบข้อสมมติฐานผ่านกราฟฟังก์ชันความหนาแน่นบนแกน Principal Component ที่ 1 จะเห็นว่าโครงสร้างของ Dataset B มีความซับซ้อน ข้อมูลในกลุ่มหลักมีการกระจายตัวที่ซ้อนทับกันอย่างรุนแรง และที่สำคัญคือแต่ละกลุ่มมีความกว้างของความแปรปรวน ที่ไม่เท่ากัน

บางกลุ่มมีความหนาแน่นกระจุกตัวสูง ในขณะที่บางกลุ่มกระจายตัวออกกว้าง อัลกอริทึม K-Means ซึ่งมีข้อจำกัดทางคณิตศาสตร์ที่มักจะสมมติให้ทุกกลุ่มเป็นทรงกลมและมีขนาดความแปรปรวนเท่าๆ กัน จึงไม่สามารถรับมือกับโครงสร้างแบบนี้ได้นัก ในทางกลับกัน GMM อาศัยหลักการความน่าจะเป็นที่อนุญาตให้กลุ่มข้อมูลมีขนาด รูปร่าง และความแปรปรวนที่ยืดหยุ่น (Soft clustering) จึงสามารถปรับตัวเข้ากับข้อมูลชุดนี้ได้ดีกว่า

หลักฐานเชิงปริมาณจากผลลัพธ์ก็ออกมายืนยันความยืดหยุ่นนี้ โดย GMM สามารถทำคะแนน ARI ได้ 0.5689 ซึ่งมากกว่า K-Means ที่ทำคะแนนได้เพียง 0.5397 หากสังเกตจากกราฟ Scatter Plot เปรียบเทียบ จะพบว่า K-Means พยายามตัดแบ่งพื้นที่ข้อมูลที่หนาแน่นด้วยขอบเขตเส้นตรงทำให้ข้อมูลกลุ่มใหญ่ถูกหั่นออกเป็นส่วนๆ ในขณะที่ GMM ใช้การประเมินจากเมทริกซ์ความแปรปรวนเกี่ยวพัน ซึ่งช่วยให้การจัดกลุ่มพื้นที่ข้อมูลเป็นวงรีที่มีความยืดหยุ่น กลมกลืนไปกับรูปร่างของกลุ่มข้อมูลที่แท้จริงได้มากกว่า GMM จึงกลายเป็นตัวเลือกที่ดีกว่าสำหรับ Dataset B เมื่อต้องระบุจำนวนกลุ่มที่แน่นอน